TAKE-HOME PROJECT WRITEUP

ShiftWave

A $0-operating-cost scheduling & timekeeping platform for a multi-location swim school

Author: Avirup Bhattacharjee
Live demo: https://shift-wave.vercel.app
Repository: github.com/Avirup26/ShiftWave
Date: June 2026

Test accounts

Role	Login
Manager (Jordan Reed)	credentials provided separately
Instructor (Avery Johnson)	credentials provided separately

This document covers: system architecture & key design decisions, the five AI-driven features, assumptions made during development, and what would come next with more time.

1. Project overview

ShiftWave is a proof-of-concept replacement for When I Work / Microsoft Teams Shifts, built for a ~23-person, multi-location swim school. Two role-gated experiences in one app: **employees** view their schedule, clock in/out with geofence verification, and request time off or shift swaps; **managers** build schedules, approve requests, review flagged punches, auto-generate schedules with AI, monitor a live dashboard, and export payroll to Gusto.

Beyond the core brief, the build adds five AI-driven features that go past "schedule + clock in/out": a natural-language schedule-editing copilot, an AI fraud/anomaly detector for timekeeping data, an AI-ranked shift-swap matchmaker, a context-aware in-app assistant, and employee-facing timecard/pay estimate pages. These are covered in detail in §4.

Scope at a glance

Area	Status	Notes
Core requirements	DONE	Login-gated schedule, geofenced clock in/out, manager editor, time-off, swaps, Gusto export
Flagship differentiators	DONE	AI auto-scheduler, punch review queue, conflict/coverage detection, manager dashboard
Beyond-spec AI features	DONE	Schedule Copilot, Anomaly Detective, Swap Matchmaker, in-app Assistant, Timecard/Pay pages
Operating cost	~$0	Firebase Spark + Vercel Hobby + Gemini free tier — no Cloud Functions, no billing account

2. Tech stack & key decisions

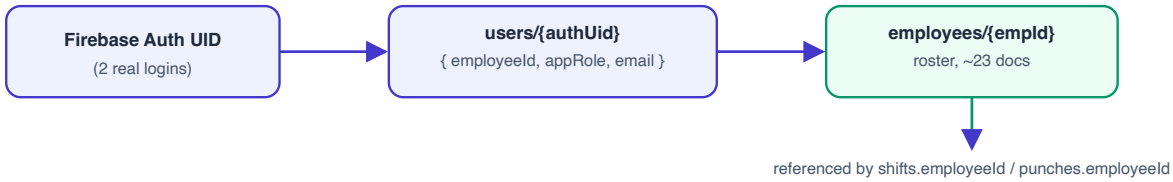
Layer	Choice	Why
Framework	Next.js 16 (App Router) + TypeScript	Server Route Handlers keep the Gemini key and Firestore Admin credentials off the client without standing up separate backend infra.
Styling	Tailwind CSS v4	Fast to ship within a tight deadline; v4's CSS-first config skips the build-tooling overhead of v3.
Auth & data	Firebase Auth + Cloud Firestore (Spark/free tier)	Matches the "Google ecosystem preferred" constraint; real-time listeners power the review queue and schedule views for free.
Server SDK	Firebase Admin SDK	Verifies ID tokens and reads role from Firestore server-side — `appRole` is deliberately not a custom claim, so it can't be spoofed via a stale token.

AI	Google Gemini (`gemini-2.5-flash`) via `@google/genai`	Server-only; used for 5 distinct features (§4), all behind structured JSON or function-calling schemas rather than free-text parsing.
Charts	recharts	Dashboard bar chart, donut chart, radial gauges.
Hosting	Vercel Hobby	Free, auto-deploys from GitHub on push to `main` .

> **Cost target hit:** Firebase Spark (no card on file) + Vercel Hobby + Gemini free tier = effectively $0/month at this team's scale. No Cloud Functions or Firebase App Hosting were used since both require a billing account.

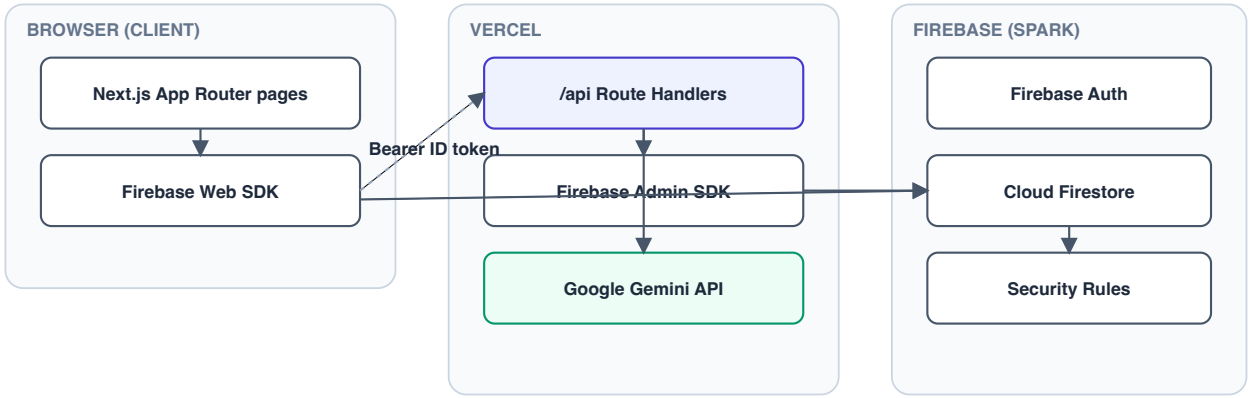
Identity model — the one decision everything else depends on

The roster and the login system are deliberately two separate Firestore collections:



Login flow: sign in → read `users/{auth.uid}` → get `employeeId` + `appRole` → load the `employees/{employeeId}` profile. Only the 2 demo accounts get real Auth logins + a `users` doc; the other ~21 staff exist only as schedulable roster records, not logins (see Assumptions, §5).

System architecture



KEY POINT:
Gemini key + Admin service account live only in Vercel env vars — never reach the browser bundle.
Client reads/writes Firestore directly via the Web SDK; rules (not just UI) enforce permissions.

Access control — three independent layers

Permissions are never enforced by a single layer:

1. **Route-group UI guards** — `(manager)/layout.tsx` requires `appRole==='manager'`; `(employee)/layout.tsx` allows any signed-in user (managers also clock in to pool shifts).
2. **Firestore Security Rules** — the real backstop. Identity resolved server-side via `get(/users/$(request.auth.uid))`; an instructor cannot read another employee's punches even with a forged client request.
3. **API route checks** — `/api/generate-schedule`, `/api/copilot`, and `/api/punch-anomalies` verify the Firebase ID token, then read `users/{uid}` via the Admin SDK to confirm `manager` (this can't be a token claim, since `appRole` isn't set at Auth-creation time) — 401 otherwise.

Shared validator core

Conflict and coverage logic — double-booking, location eligibility, over-hours, understaffed-shift detection — lives once in `src/lib/validators.ts` as pure functions with no Firebase or React imports. The schedule editor calls it live on every edit; the AI auto-scheduler runs it as a post-generation repair/validation pass; the Copilot runs it on its proposed diff before showing it to the manager; the dashboard reuses it for coverage-gap counts. One source of truth for "is this schedule valid," reused by four different surfaces.

3. Core feature walkthrough

Page	Who	What it does
`/schedule`	Employee	Real-time list of the signed-in user's shifts for the selected week, grouped by day.
`/clock`	Employee	Pick an upcoming shift, clock in (captures geolocation, computes Haversine distance to the site, flags Inside/Outside Geofence + On Time/Outside Window), clock out later. Anything outside geofence or timing window auto-flags `Needs Review`.
`/timecard`	Employee	Day-by-day worked-hours breakdown from approved punches, with Regular/Overtime split at 40h/week.
`/pay`	Employee	Simulated take-home pay (gross – flat-rate tax estimate) for the selected week, clearly labeled as a demo estimate.
`/requests`	Employee	Submit time-off (date range + reason) or a shift-swap (pick a shift + proposed replacement, with an optional AI-ranked suggestion — see §4).
`/dashboard`	Manager	KPI cards + recharts: hours/employee, overtime risk, labor-cost estimate, coverage gaps, review-queue/coverage gauges.
`/schedule-editor`	Manager	Week grid (locations × days). Add/edit/delete shifts with live conflict & coverage warnings; "Generate week with AI" and the Copilot bar (§4) both write into this same validated state.
`/review-queue`	Manager	Real-time inbox of flagged punches with distance-from-site and flag reason; one-tap Approve/Reject.

`/approvals`	Manager	Approve/deny pending time-off and swap requests; approving a swap reassigns the shift's `employeeId`.
`/payroll`	Manager	Gusto-format CSV export, one row per **approved** punch, matching the sample workbook's exact column order.
`/insights`	Manager	AI Anomaly Detective (§4) — on-demand scan for suspicious punch patterns.

4. The five AI features

Every AI call follows the same architectural rule: **the model never has unchecked authority.** It either returns structured JSON/function-calls that pass through the deterministic validator before a human sees them, or it answers questions using only data explicitly handed to it in context — never inventing figures.

1. AI Auto-Scheduler `/schedule-editor → "Generate week with AI"`

Gemini drafts a full week of shift assignments from coverage rules, eligibility, target hours, and approved time-off, returning JSON only. The draft is then run through `validateSchedule()` — the same pure function the editor uses for live warnings — which auto-repairs anything it safely can (drops ineligible/over-hours assignments) and surfaces remaining issues instead of hiding them. The manager reviews a diff modal and explicitly accepts before anything is written to Firestore. *Pattern: LLM proposes → deterministic validator repairs/flags → human accepts.*

2. Manager Copilot `/schedule-editor → natural-language bar`

A manager types a plain-English instruction ("move Avery's Tuesday shift to Thursday," "add an Ambassador at Mansfield Friday night"). Gemini is given *function-calling* tools — `add_shift`, `remove_shift`, `reassign_shift`, `move_shift` — scoped to only the current week's real shift/employee/location IDs (it cannot invent an ID). The resolved operations are applied to a working copy of the week, validated, and shown as a before/after diff before the manager applies them. This is editing-by-conversation on top of the exact same validator the grid view uses.

3. Punch Anomaly Detective `/insights`

A deterministic signal layer (`computeAnomalySignals`) first computes hard facts per employee — geofence violation rate, average clock-in delta vs. scheduled start, early-clock-in count, max distance from site — and detects **buddy-punch clusters** (≥2 employees clocking in within 3 minutes of each other at the same site, both outside the geofence — a classic sign one person is punching in for another). Only if real signal exists is Gemini asked to rank and explain the concerning patterns, grounded strictly in the precomputed numbers ("do not invent figures"). This keeps the model from hallucinating fraud where none exists — it explains evidence, it doesn't generate it.

4. AI Swap Matchmaker `/requests → "Suggest best replacement"`

When an employee wants to swap a shift, `buildSwapCandidates()` scores every eligible coworker on hard facts: location eligibility, projected weekly hours after the swap, double-booking conflicts, and approved time-off overlap. Gemini ranks the shortlist and writes a one-line, human rationale for each — but the eligibility and conflict math is 100% deterministic; the model only adds judgment and language on top of facts it can't get wrong.

5. In-app AI Assistant `site-wide chat widget`

A conversational widget available to any signed-in user. Each request is scoped server-side to that user's own data only (their shifts, hours, pending requests this week) — an employee's assistant session never sees another employee's data, and only managers get an aggregate snapshot (counts of punches needing review, pending requests). It can also compute simulated take-home pay on the fly using the same formula as the `/pay` page, so "what would I make this week?" gets a real number, not a deflection. Conversation resets automatically on account switch so sessions never leak across users.

> **Why this matters as a portfolio signal:** all five features share one architectural discipline — compute the facts deterministically first, then use the LLM only for ranking / natural-language translation / explanation on top of facts that are already correct. None of the five trust Gemini to get arithmetic, eligibility, or IDs right on its own.

5. Assumptions made during development

Assumption	Reasoning
Two access roles only: `manager` / `employee`	Ambassadors and Instructors are both modeled as `employee` — the sample data has no finer-grained permission tiers.
Only 2 real logins; ~21 other staff are roster-only	The brief didn't provide credentials for 23 people. Modeling everyone as a schedulable roster entity (referenced by shifts/punches) but only provisioning real Firebase Auth accounts for the 2 demo users keeps the POC realistic without 23 fake email addresses.
Weekly pay period, Mon–Sun; overtime > 40h/week	Standard FLSA/Texas default — not specified in source data.
Geofence: 200ft pools, 300ft events, none remote; 5-min on-time window	Radius is a reasonable default for a parking-lot-sized facility; the 5-minute window was reverse-engineered from the seeded sample punches' actual on-time/late labels.
Client-side geofencing	Acceptable for a POC; flagged explicitly as a limitation (§6) — production would re-verify server-side and add App Check to prevent location spoofing.

Hourly pay rates are assumed, not sourced	Not present in the sample data. Used only for the dashboard's labor-cost *estimate* and the demo `/pay` page — both UI-labeled as estimates, never used for the actual Gusto export.
Only manager-approved punches count toward payroll	Protects against silently paying disputed/unverified time; the export surfaces excluded punches with a link back to the review queue rather than dropping them silently.
Demo week fixed to 2026-06-22	The seeded schedule lives in one specific week; all date pickers default there (with a "Demo week" jump button) so the app never looks empty regardless of the real calendar date.
Events have no per-occurrence coordinates	The sample sheet leaves event lat/lng blank; event clock-ins are treated as `No Geofence` rather than guessing a location.
PTO is excluded from the Gusto CSV	The sample `GustoExportSample` sheet has no PTO column; approved time off is tracked in `/approvals` and would map to a separate Gusto PTO import in production — deliberately scoped out, not missing.
Gemini model pinned to `gemini-2.5-flash`	Current fast/cheap model as of build time; isolated as a single constant for a one-line swap when Google deprecates it.

6. Security & data model summary

Collection	Read	Write
`users/{uid}`	Own doc, or any manager	Admin SDK only (never client-writable)
`employees`, `locations`, `roles`, `shifts`	Any signed-in user	Manager only
`punches`	Own punches, or manager (all)	Create own (cannot self-approve on create); manager updates `managerReviewStatus`
`timeOffRequests` / `swapRequests`	Own, or manager	Create as requester; manager updates status

Rules resolve identity via `get(/users/$(request.auth.uid))` — never trust a client-supplied role field.

7. What I'd build next, given more time

Item	Why it's next, not now
Server-side geofence validation + Firebase App Check	Client-side geocoordinates can be spoofed via dev tools; this is the single biggest gap between POC and production-grade timekeeping integrity.

Push / email notifications	Manager approvals and review-queue items currently require a manual visit to the page; notifications would close the loop.
Google SSO	Drop password management entirely, consistent with the "Google ecosystem" preference in the brief.
Google Sheets / Calendar sync	Two-way sync would let managers work from a spreadsheet for bulk edits and give employees a calendar feed of their shifts.
Recurring shift templates	The editor and AI scheduler currently build one week at a time; a "repeat this week" template would speed up steady-state scheduling.
Audit log + payroll-period locking	Once a pay period is exported to Gusto, punches in that range should become immutable with a change history — not currently enforced.
Direct Gusto API integration	The CSV export matches Gusto's import format exactly; a direct API push would remove the manual upload step.
Native mobile wrapper	For reliable background geofencing and push notifications beyond what a PWA can do in the browser.
Extend the Copilot to time-off/swap approvals	Currently scoped to shift add/remove/reassign/move; "approve all of Avery's pending requests" is a natural extension of the same function-calling pattern.
Real payroll tax integration for `/pay`	Currently a flat-rate simulation labeled as an estimate; a real bracket/jurisdiction-aware calculation (or a Gusto paystub API call) would make it accurate, not illustrative.